

Apigee Policies: A Simple Guide with Real-Time Examples

Apigee policies are like special rules or tools that you can attach to your APIs to control how they behave. They help you manage traffic, keep your APIs secure, and transform data. Think of them as building blocks that add powerful features to your APIs without writing a lot of code.

Here's a simplified look at common Apigee policies, systematically organized by their function, along with real-time examples:

1. Traffic Management Policies

These policies help you control the flow of requests to your APIs, ensuring they don't get overloaded and remain available.

a. Quota Policy

- **What it does:** The Quota policy sets a limit on the number of API requests an app developer can make to your API within a specific time period (e.g., 1000 requests per month). It's like a monthly data plan for your phone.
- **Why it's useful:** Prevents abuse, ensures fair usage among different app developers, and helps manage your backend resources.
- **Real-time Example:** Imagine you have a weather API. You offer a free tier that allows a mobile app developer to make up to 1,000 weather forecast requests per day. Once their app hits this limit, the Quota policy will block further requests from that app until the next day. This ensures that a single free app doesn't consume all your API resources, keeping the service available for everyone.

b. Spike Arrest Policy

- **What it does:** The Spike Arrest policy protects your backend services from sudden, unexpected bursts of traffic. It smooths out traffic spikes by rejecting requests that exceed a defined rate (e.g., 30 requests per second). Think of it as a bouncer at a popular club, letting people in at a steady pace even if a huge crowd arrives all at once.
- **Why it's useful:** Prevents your backend servers from crashing due to sudden traffic surges, which could be caused by a viral event, a bot attack, or a sudden marketing campaign.
- **Real-time Example:** Let's say your e-commerce API processes orders. During a flash sale, thousands of users might try to place orders simultaneously. A Spike Arrest policy configured to allow, say, 500 orders per second would temporarily reject requests beyond that rate, preventing your order processing system from being overwhelmed and crashing. Users might get a "try again later" message, but the system remains stable and can process the valid requests.

c. Caching Policies

Caching policies help improve API performance and reduce the load on your backend services by storing frequently accessed data closer to the client.

i. Response Cache Policy

- **What it does:** This policy stores responses from your backend service for a specified period. When a subsequent request for the same data comes in, Apigee can serve the cached response directly, without needing to call the backend again.
- **Why it's useful:** Significantly speeds up API response times for frequently accessed data, reduces latency, and decreases the load on your backend servers, saving resources and costs.
- **Real-time Example:** Imagine you have an API that provides daily weather forecasts. The weather forecast for a specific city doesn't change every second. You can configure a Response Cache policy to store the forecast for 10 minutes. If 100 different users request the weather for London within that 10-minute window, only the first request will hit your backend weather service. The

subsequent 99 requests will be served instantly from Apigee's cache, making your API much faster and reducing the burden on your weather data provider.

ii. Invalidate Cache Policy

- **What it does:** This policy allows you to explicitly remove (invalidate) entries from the cache. This is crucial when the data that was cached has changed on the backend and you want to ensure users get the most up-to-date information.
- **Why it's useful:** Ensures data freshness. While caching improves performance, serving stale data can be problematic. This policy gives you control to clear the cache when necessary.
- **Real-time Example:** Continuing with the weather API example, if there's a sudden, unexpected change in weather (e.g., a tornado warning issued), your backend weather service might update its data immediately. To ensure your API users get this critical, updated information without waiting for the cache to expire naturally, you can trigger an Invalidate Cache policy. This policy would immediately clear the cached weather forecast for that city, forcing the next request to fetch the fresh data from your backend.

2. Security Policies

Security policies are vital for protecting your APIs from unauthorized access and malicious attacks. They act as gatekeepers and validators, ensuring only legitimate and safe requests reach your backend.

a. API Key Verification Policy

- **What it does:** This policy checks if an incoming API request includes a valid API key. An API key is a unique code given to each app developer. If the key is missing or invalid, the request is rejected.
- **Why it's useful:** It's the simplest way to identify who is using your API and to control access. You can track usage per key and easily revoke access for specific apps if needed.
- **Real-time Example:** Imagine you have an API that provides stock market data. Before any app can get data, it must present a valid API key. If a financial news

app tries to access your API without a key, or with an expired key, the API Key Verification policy will immediately deny the request. This ensures only registered and authorized applications can consume your valuable data.

b. OAuthV2 Policy (for OAuth 2.0)

- **What it does:** The OAuthV2 policy handles the OAuth 2.0 authorization framework, which is a more secure way for applications to access user data without needing their username and password. It issues and verifies access tokens.
- **Why it's useful:** Provides secure, delegated access. Users grant permission to applications to access their data on their behalf, without sharing their credentials directly with the application. This is crucial for protecting sensitive user information.
- **Real-time Example:** Consider a social media API. When a third-party photo editing app wants to access a user's photos from their social media account, the OAuthV2 policy comes into play. Instead of the photo app asking for the user's social media password, it redirects the user to the social media platform to log in and grant permission. The social media platform then issues an access token to the photo app. The photo app uses this token to access the user's photos. The OAuthV2 policy in Apigee verifies this token with every request, ensuring the photo app only accesses what it's authorized to, and the user's password is never exposed to the photo app.

c. Decode JWT Policy

- **What it does:** This policy parses a JSON Web Token (JWT) and makes its claims (the information inside the token) available as flow variables within Apigee. It doesn't verify the signature of the JWT, only decodes its contents.
- **Why it's useful:** Allows you to inspect the contents of a JWT early in the API proxy flow, even before full verification, to make routing decisions or log information based on the token's claims.
- **Real-time Example:** A mobile banking app sends a JWT with each request, containing user ID and role. Before processing the request, you can use a Decode JWT policy to extract the user ID and role. Based on the user's role (e.g., 'admin')

vs. 'customer'), you might route the request to different backend services or apply different rate limits, all before fully verifying the token's authenticity.

d. Verify JWT Policy

- **What it does:** This policy validates the signature of a JWT to ensure it hasn't been tampered with and that it was issued by a trusted party. It also checks for token expiration and other claims.
- **Why it's useful:** Essential for ensuring the integrity and authenticity of tokens used for authentication and authorization. It's a critical security measure.
- **Real-time Example:** After decoding the JWT from the banking app, you use a Verify JWT policy. This policy checks if the token's signature is valid using a public key you trust. If the signature is invalid (meaning the token was altered) or if the token has expired, the policy will reject the request, preventing unauthorized access to sensitive banking operations.

e. Basic Authentication Policy

- **What it does:** This policy enforces HTTP Basic Authentication, where a username and password (encoded in Base64) are sent in the `Authorization` header of an HTTP request. Apigee verifies these credentials against a configured list or an external identity store.
- **Why it's useful:** A simple and widely understood authentication mechanism, often used for internal APIs or when integrating with legacy systems that don't support more modern authentication methods like OAuth.
- **Real-time Example:** You have an internal API that allows your sales team to update customer records. This API is not exposed publicly. You can secure it with a Basic Authentication policy. Each sales application needs to send a username and password (e.g., `sales_user:password123`) with every request. If the credentials don't match what's configured in Apigee, the request is denied, ensuring only authorized internal tools can modify customer data.

f. JSON Threat Protection Policy

- **What it does:** This policy protects your API from JSON-based attacks by validating the structure and content of incoming JSON payloads against predefined limits (e.g., maximum string length, maximum array elements, maximum object depth).
- **Why it's useful:** Prevents attacks like JSON bomb attacks (extremely large and nested JSON payloads designed to consume server resources) or other malformed JSON that could exploit vulnerabilities in your backend.
- **Real-time Example:** Your API accepts JSON data for user profiles. A malicious actor might try to send a JSON payload with an extremely long string in the 'bio' field or an array with millions of elements, hoping to crash your server. A JSON Threat Protection policy can be configured to reject any JSON payload where a string exceeds 255 characters or an array has more than 100 elements, effectively blocking such attacks.

g. XML Threat Protection Policy

- **What it does:** Similar to JSON Threat Protection, this policy protects your API from XML-based attacks by validating the structure and content of incoming XML payloads against predefined limits (e.g., maximum element depth, maximum attribute count, maximum entity expansion).
- **Why it's useful:** Guards against XML bomb attacks (e.g., Billion Laughs attack) or other malformed XML that could lead to denial-of-service (DoS) or other vulnerabilities in your backend systems.
- **Real-time Example:** If your API processes XML requests, a malicious user could send an XML payload designed to cause excessive memory consumption through entity expansion. An XML Threat Protection policy can be set to limit the number of entity expansions or the depth of the XML structure, preventing your API from being exploited by such attacks.

h. Regular Expression Protection Policy

- **What it does:** This policy allows you to validate input (like query parameters, headers, or message body content) against a regular expression pattern. If the

input doesn't match the pattern, the request can be rejected.

- **Why it's useful:** Provides fine-grained control over input validation, helping to prevent injection attacks (SQL injection, XSS) and ensuring data quality before it reaches your backend.
- **Real-time Example:** Your API has a query parameter `zipCode`. You want to ensure that users only enter valid 5-digit or 5-digit-plus-4-digit zip codes. You can use a Regular Expression Protection policy to validate `zipCode` against a regex pattern like `^\d{5}(-\d{4})?$`. If a user enters `12345abc` or `123`, the policy will reject the request, ensuring only correctly formatted zip codes are processed.

i. Access Control Policy

- **What it does:** This policy allows you to control access to your API based on the IP addresses of the incoming requests. You can either allow or deny requests from specific IP addresses or ranges.
- **Why it's useful:** Provides a network-level security layer, useful for restricting API access to known networks (e.g., your corporate VPN) or blocking known malicious IP addresses.
- **Real-time Example:** You have an administrative API that should only be accessible from your company's internal network. You can configure an Access Control policy to only allow requests originating from your company's specific IP address range. Any request coming from outside this range will be automatically denied, even if it has valid authentication credentials, adding an extra layer of security.

3. Mediation Policies

Mediation policies allow you to change or transform the format and content of messages (requests and responses) as they pass through your API proxy. They act like translators or editors, adapting data to meet different system requirements.

a. JSON to XML Policy (and vice-versa)

- **What it does:** This policy converts the format of the message payload from JSON (JavaScript Object Notation) to XML (Extensible Markup Language), or from XML to JSON. This is super useful when your backend service expects a different data format than what your client application sends.
- **Why it useful:** It bridges the gap between different systems. For example, a modern mobile app might send data in JSON, but an older backend system might only understand XML. This policy handles the conversion seamlessly, allowing disparate systems to communicate without requiring changes to either.
- **Real-time Example:** Imagine a new mobile app that sends user registration data to your API in JSON format: `{"username": "john.doe", "email": "john@example.com"}`. However, your legacy user management system (the backend) only accepts user data in XML format: `<User><Username>john.doe</Username><Email>john@example.com</Email></User>`. You can attach a JSON to XML policy to your API proxy. When the mobile app sends a JSON request, Apigee automatically converts it to XML before forwarding it to your backend, and vice-versa for responses. The mobile app developer doesn't even know the backend uses XML!

b. Assign Message Policy

- **What it does:** The Assign Message policy is a very versatile policy that allows you to create, modify, or delete parts of a request or response message. You can add new headers, change existing ones, set query parameters, or even create an entirely new message.
- **Why it useful:** It helps you customize messages on the fly. For instance, you might want to add a special header to all requests going to your backend, or remove sensitive information from a response before sending it back to the client. This policy is fundamental for message manipulation and ensuring messages conform to specific formats or contain necessary information.
- **Real-time Example:** Let's say your backend API requires a specific `X-API-Key` header for internal authentication, but your public-facing API doesn't expose this to client apps. You can use an Assign Message policy in Apigee to *add* this `X-API-key` header with a predefined value to every request *before* it reaches your

backend. So, the client sends a simple request, and Apigee enriches it with the necessary internal header. Similarly, you could use it to remove an internal `x-Debug-Info` header from the response before it goes back to the client, preventing internal details from being exposed.

4. Data Manipulation and Storage Policies

These policies help you work with data within your API proxy, extracting information or storing configuration data.

a. Extract Variables Policy

- **What it does:** This policy allows you to extract specific pieces of information from a request or response message (like headers, query parameters, path parameters, or parts of the JSON/XML payload) and store them as variables within Apigee. These variables can then be used by other policies or for conditional logic.
- **Why it useful:** It helps you pull out important data from messages to use it later in your API proxy flow. This is fundamental for dynamic behavior and data-driven decisions.
- **Real-time Example:** Your API receives a request with a JSON body containing user details, including a `user_id`. You need to use this `user_id` to look up additional information in a KVM (Key Value Map) or to log it. An Extract Variables policy can be configured to read the `user_id` from the JSON payload and save it into a new flow variable, say `extracted.user_id`. Later policies can then easily access `extracted.user_id` without needing to re-parse the entire JSON message.

b. Key Value Map Operations (KVM) Policy

- **What it does:** The KVM policy allows you to store, retrieve, update, or delete data in a Key Value Map (KVM) within Apigee. KVMs are like simple databases for storing configuration data, API keys, or other small pieces of information that your API proxies need to access quickly.
- **Why it useful:** Provides a way to store and retrieve configuration data that can be easily updated without redeploying your API proxy. This is great for dynamic

settings, feature flags, or storing non-sensitive credentials.

- **Real-time Example:** You have different API rate limits for various API products, and these limits might change frequently. Instead of hardcoding these limits in your policies, you can store them in a KVM. A KVM policy can then retrieve the current rate limit for a specific API product (e.g., `get.rate_limit_for_product_A`) and use that value in a Quota policy. This way, you can update the rate limits in the KVM without touching or redeploying your API

5. Extension Policies

Extension policies allow you to add custom logic and behavior to your API proxies using scripting languages. This is useful when the built-in policies don't quite cover your specific needs.

a. JavaScript Policy

- **What it does:** This policy allows you to execute custom JavaScript code within your API proxy flow. You can use JavaScript to access and manipulate flow variables, modify request/response messages, call external services, or implement complex logic.
- **Why it useful:** It provides immense flexibility. If you have a very specific requirement that no other policy can handle, you can write custom JavaScript to achieve it. It's like having a mini-program running inside your API proxy.
- **Real-time Example:** Let's say your API needs to dynamically route requests to different backend servers based on a complex calculation involving multiple request headers and query parameters. Or perhaps you need to generate a unique transaction ID that follows a very specific format. You can write a JavaScript policy to perform these calculations, manipulate the message, and set a flow variable that a subsequent policy can use for routing. For instance, you could use JavaScript to check if a user is from a specific region and, if so, add a custom header to the request before it reaches the backend, indicating that it should be handled by a regional server.

6. Integration and Utility Policies

These policies facilitate communication with external services, organize policy execution, and help with logging and debugging.

a. Service Callout Policy

- **What it does:** This policy allows your API proxy to make a call to an external service (another API, a microservice, or any HTTP endpoint) during the API proxy flow. It can be used to fetch additional data, perform validations, or enrich the request/response.
- **Why it useful:** Enables your API proxy to integrate with other services in real-time. This is powerful for fetching dynamic data, performing lookups, or triggering external processes without involving the original backend target.

b. Flow Callout Policy

- **What it does:** This policy allows you to execute a shared flow from within your API proxy. Shared flows are reusable sequences of policies that can be called from multiple API proxies, promoting modularity and reusability.
- **Why it useful:** Helps you avoid duplicating policies across multiple API proxies. If you have a common set of policies (e.g., for logging, error handling, or authentication) that you want to apply to many APIs, you can put them in a shared flow and call it using the Flow Callout policy.
- **Real-time Example:** You have a standard set of security checks (API Key verification, OAuth token validation, JSON threat protection) that every API in your organization must perform. Instead of adding these three policies to every single API proxy, you can create a shared flow containing these policies. Then, in each API proxy, you simply use a Flow Callout policy to execute this shared security flow. If you need to update the security checks, you only change the shared flow, and all APIs using it automatically get the update.

c. Message Logging Policy

- **What it does:** This policy allows you to log message data (like request headers, response payloads, or flow variables) to an external logging service (e.g., Splunk,

Loggly, or a custom log server). It helps in monitoring, debugging, and auditing API traffic.

- **Why it useful:** Provides visibility into your API traffic and helps you troubleshoot issues, analyze usage patterns, and meet compliance requirements. It captures important information about each API call.
- **Real-time Example:** To monitor API performance and debug errors, you want to log every incoming request and outgoing response, including timestamps, client IP addresses, and any error messages. You can attach a Message Logging policy to your API proxy to send this information to your centralized logging system. If a user reports an issue, you can then easily search your logs to see the exact request they sent, the response they received, and any errors that occurred during processing.